

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Pseudocode Design

Course Code:	DSA0404	Course Title:	FUNDAMENTALS OF DATA SCIENCE
CO Assessed:	CO5 – Pseudocode Design	Assessment:	Assessment Tool 1– Pseudocode Design
Weightage:	30% of CIA	Total Marks:	100
CO3 Statement:	To understand the concept of supervised and unsupervised methods in machine learning		
Student Name:	Deva Dharshini	Reg. No.:	192472350
Section / Batch:	BE-CSE-AI,2024	Date:	24-08-26

Code of Conduct

I S. Deva Dharshini(192472350) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Deva Dharshini

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly	
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts	
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning	
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided	
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand	

1. Design pseudocode to implement the K-Means clustering algorithm: initialize centroids, assign observations to the nearest centroid, update centroids, and repeat until convergence.

Problem Statement

Design and implement the K-Means clustering algorithm to group observations into a specified number of clusters. The algorithm should initialize centroids, assign observations to the nearest centroid, update the centroids, and repeat the process until convergence.

Algorithm

1. Start.
2. Import the required libraries.
3. Create or load the dataset.
4. Select the number of clusters K.
5. Initialize K centroids.
6. Calculate the distance between each observation and all centroids.
7. Assign each observation to the nearest centroid.
8. Calculate new centroids using the mean of the observations in each cluster.
9. Repeat the assignment and update process until convergence.
10. Display the cluster labels and centroids.
11. Stop.

Pseudocode

BEGIN

INPUT dataset X

INPUT number of clusters K

Initialize K centroids

REPEAT

FOR each observation x in X

Calculate distance from x to every centroid

Assign x to the nearest centroid

END FOR

FOR each cluster

 Calculate the mean of all observations

 Update the centroid

END FOR

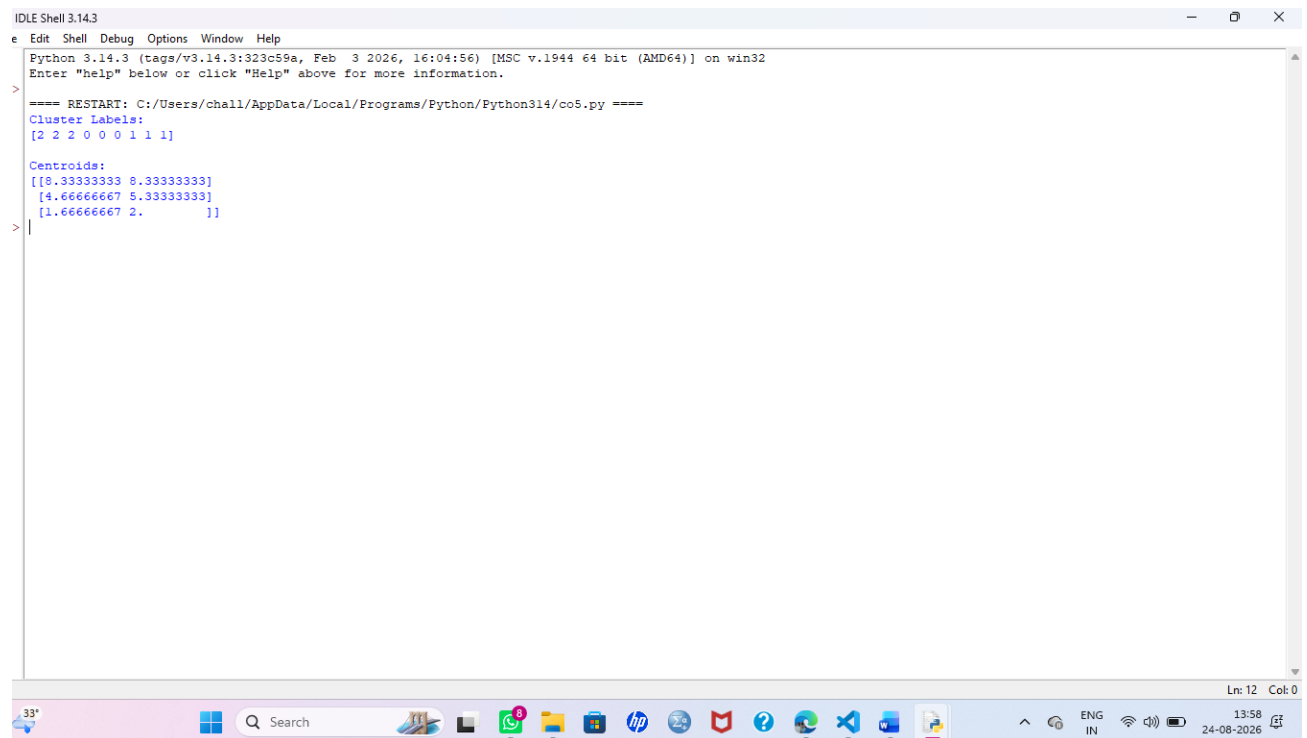
UNTIL centroids do not change

DISPLAY cluster labels

DISPLAY final centroids

END

Output



```
IDLE Shell 3.14.3
Python 3.14.3 (tags/v3.14.3:323c59a, Feb  3 2026, 16:04:56) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>
==== RESTART: C:/Users/chall/AppData/Local/Programs/Python/Python314/co5.py ====
Cluster Labels:
[2 2 2 0 0 0 1 1]

Centroids:
[[8.33333333 8.33333333]
 [4.66666667 5.33333333]
 [1.66666667 2.        ]]
```

2. Design pseudocode to group customers into three clusters based on annual income and spending score using K-Means clustering.

Problem Statement

Design and implement a K-Means clustering algorithm to group customers into three different clusters based on their Annual Income and Spending Score.

Algorithm

1. Start.
2. Import the required libraries.
3. Create or load customer data.
4. Select Annual Income and Spending Score as features.
5. Set the number of clusters as $K = 3$.
6. Initialize three centroids.
7. Calculate the distance between each customer and the centroids.
8. Assign each customer to the nearest cluster.
9. Update the centroids.
10. Repeat until convergence.
11. Display the customer clusters.
12. Stop.

Pseudocode

BEGIN

INPUT customer data

Select Annual Income and Spending Score

SET $K = 3$

Initialize 3 centroids

REPEAT

FOR each customer

 Calculate distance to each centroid

 Assign customer to nearest centroid

END FOR

FOR each cluster

 Calculate mean Annual Income

 Calculate mean Spending Score

 Update centroid

END FOR

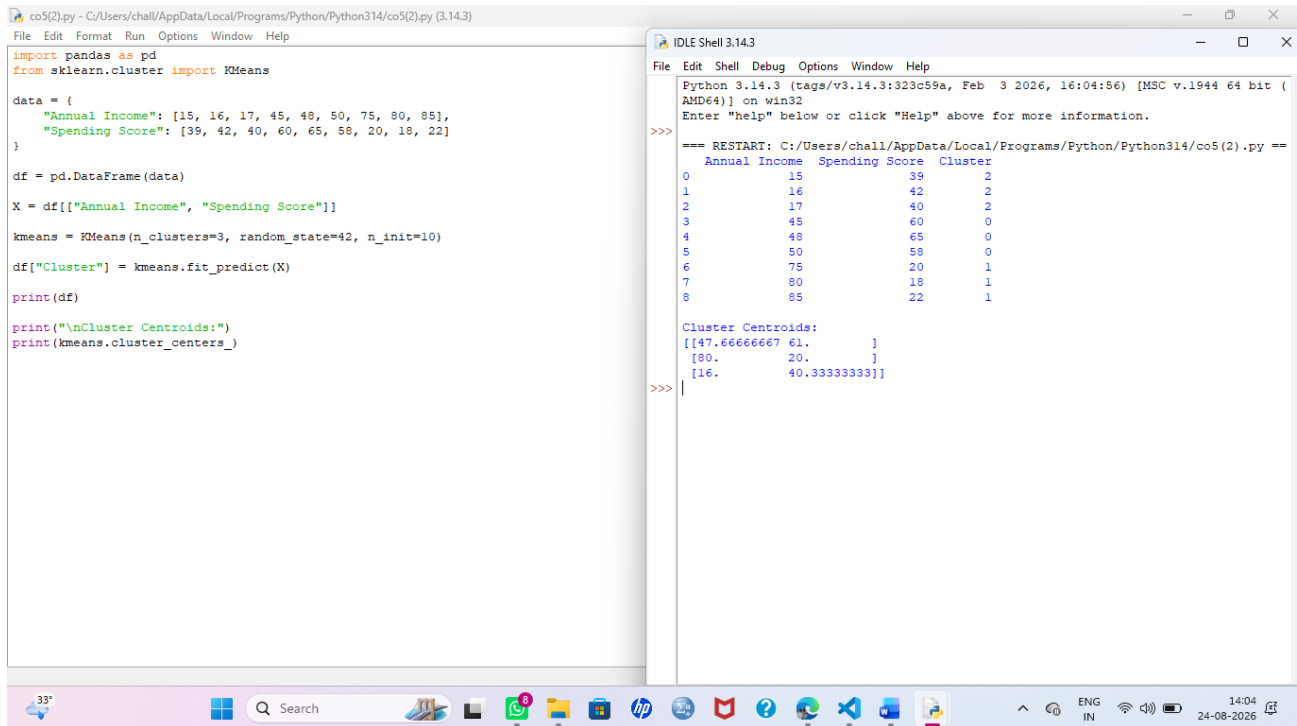
UNTIL centroids do not change

DISPLAY customer clusters

DISPLAY cluster centroids

END

Output



The screenshot shows a Python IDE with two windows. The left window displays the code for K-Means clustering, and the right window shows the output.

```
co5(2).py - C:/Users/chall/AppData/Local/Programs/Python/Python314/co5(2).py (3.14.3)
File Edit Format Run Options Window Help

import pandas as pd
from sklearn.cluster import KMeans

data = {
    "Annual Income": [15, 16, 17, 45, 48, 50, 75, 80, 85],
    "Spending Score": [39, 42, 40, 60, 65, 58, 20, 18, 22]
}

df = pd.DataFrame(data)
X = df[["Annual Income", "Spending Score"]]

kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
df["Cluster"] = kmeans.fit_predict(X)
print(df)

print("\nCluster Centroids:")
print(kmeans.cluster_centers_)
```

The output window shows the following results:

```
Python 3.14.3 (tags/v3.14.3:323c59a, Feb 3 2026, 16:04:56) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
=== RESTART: C:/Users/chall/AppData/Local/Programs/Python/Python314/co5(2).py ===
  Annual Income  Spending Score  Cluster
0             15             39         2
1             16             42         2
2             17             40         2
3             45             60         0
4             48             65         0
5             50             58         0
6             75             20         1
7             80             18         1
8             85             22         1

Cluster Centroids:
[[47.66666667 61.        ]
 [80.        20.        ]
 [16.        40.33333333]]
>>>
```

3. Design pseudocode to determine the suitable number of clusters using the Elbow Method and then apply K-Means with the selected number of clusters.

Problem Statement

Design and implement the Elbow Method to determine the suitable number of clusters for a dataset. Apply K-Means clustering using the selected number of clusters.

Algorithm

1. Start.
2. Import the required libraries.
3. Load the dataset.
4. Select a range of values for K.
5. Apply K-Means for each value of K.

6. Calculate the WCSS value.
7. Store all WCSS values.
8. Plot the Elbow graph.
9. Identify the suitable value of K.
10. Apply K-Means using the selected value.
11. Display the cluster labels.
12. Stop.

Pseudocode

BEGIN

INPUT dataset X

FOR K = 1 to maximum number of clusters

 Apply K-Means with K clusters

 Calculate WCSS

 Store WCSS value

END FOR

Plot K versus WCSS

Identify elbow point

SET selected_K = elbow point

Apply K-Means using selected_K

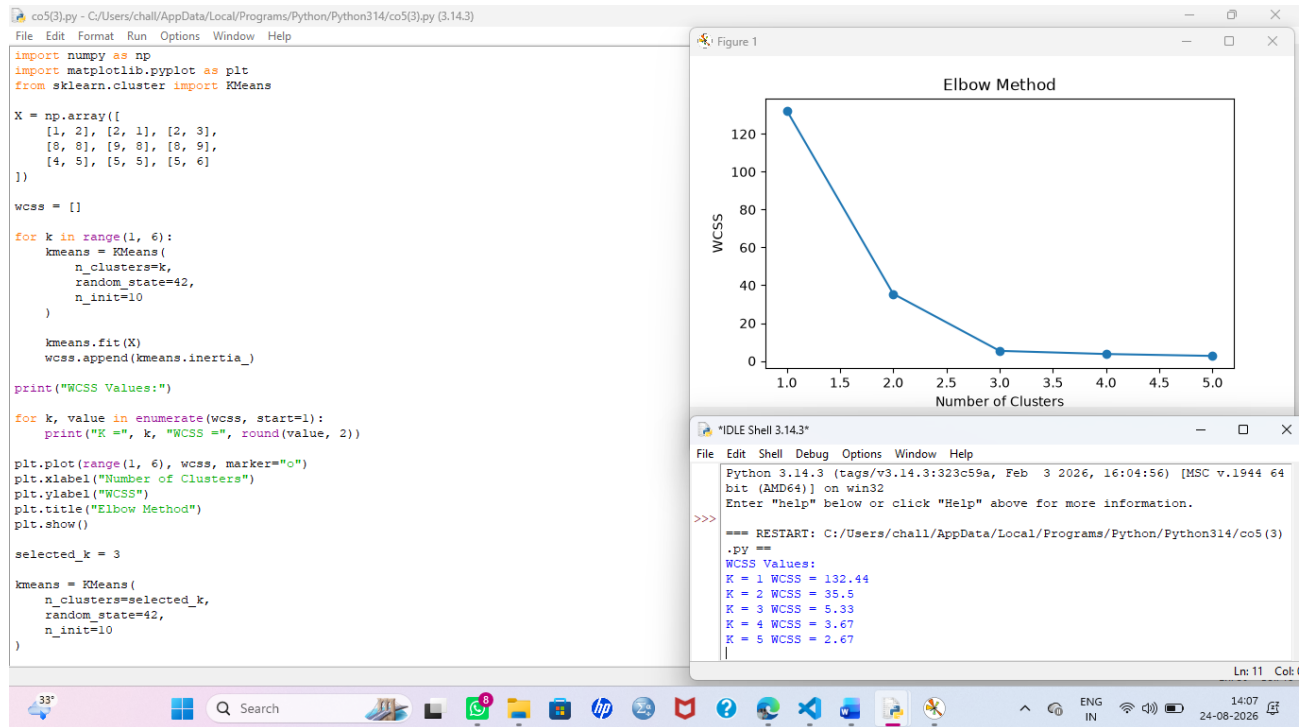
Assign observations to clusters

DISPLAY selected_K

DISPLAY cluster labels

END

Output



4. Design pseudocode to calculate TP, TN, FP, and FN from actual and predicted binary class labels and subsequently calculate accuracy, precision, recall, and F1-score.

Problem Statement

Design and implement a program to calculate True Positive, True Negative, False Positive, and False Negative from actual and predicted binary class labels. Calculate Accuracy, Precision, Recall, and F1-Score using these values.

Algorithm

1. Start.
2. Import the required libraries.
3. Define actual class labels.
4. Define predicted class labels.
5. Calculate TP, TN, FP, and FN.
6. Calculate Accuracy.
7. Calculate Precision.
8. Calculate Recall.
9. Calculate F1-Score.

10. Display all the values.

11. Stop.

Pseudocode

BEGIN

INPUT actual labels

INPUT predicted labels

SET TP = 0

SET TN = 0

SET FP = 0

SET FN = 0

FOR each actual and predicted value

IF actual = 1 AND predicted = 1

TP = TP + 1

ELSE IF actual = 0 AND predicted = 0

TN = TN + 1

ELSE IF actual = 0 AND predicted = 1

FP = FP + 1

ELSE IF actual = 1 AND predicted = 0

FN = FN + 1

END IF

END FOR

Accuracy = $(TP + TN) / (TP + TN + FP + FN)$

Precision = $TP / (TP + FP)$

Recall = $TP / (TP + FN)$

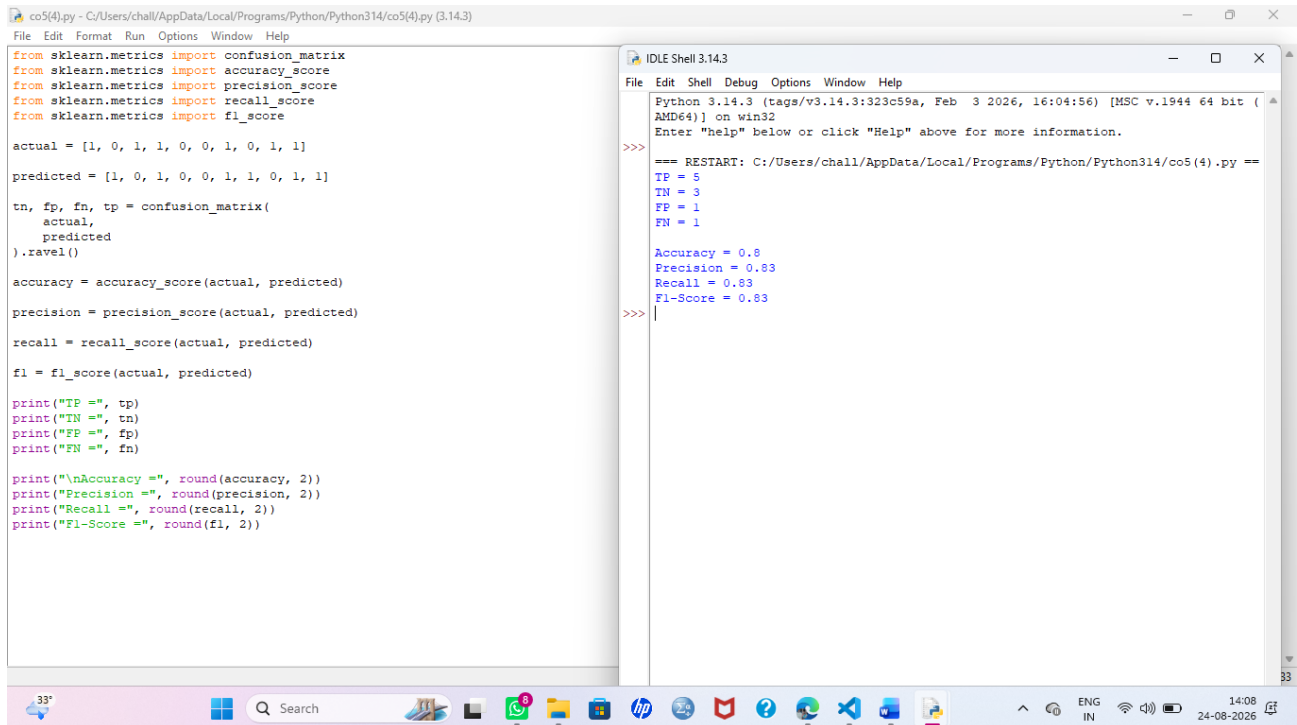
F1_Score = $2 \times \text{Precision} \times \text{Recall} / (\text{Precision} + \text{Recall})$

DISPLAY TP, TN, FP, FN

DISPLAY Accuracy, Precision, Recall, F1_Score

END

Output



The screenshot shows a Python IDE with two windows. The left window displays a script named `co5(4).py` that imports metrics from `sklearn.metrics` and calculates confusion matrix values for two classes. The right window shows the IDLE Shell output, which includes the file path, Python version, and the calculated metrics.

```
co5(4).py - C:/Users/chall/AppData/Local/Programs/Python/Python314/co5(4).py (3.14.3)
File Edit Format Run Options Window Help

from sklearn.metrics import confusion_matrix
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score

actual = [1, 0, 1, 1, 0, 0, 1, 0, 1, 1]
predicted = [1, 0, 1, 0, 0, 1, 1, 0, 1, 1]

tn, fp, fn, tp = confusion_matrix(
    actual,
    predicted
).ravel()

accuracy = accuracy_score(actual, predicted)
precision = precision_score(actual, predicted)
recall = recall_score(actual, predicted)
f1 = f1_score(actual, predicted)

print("TP =", tp)
print("TN =", tn)
print("FP =", fp)
print("FN =", fn)

print("\nAccuracy =", round(accuracy, 2))
print("Precision =", round(precision, 2))
print("Recall =", round(recall, 2))
print("F1-Score =", round(f1, 2))

IDLE Shell 3.14.3
File Edit Shell Debug Options Window Help

Python 3.14.3 (tags/v3.14.3:323c59a, Feb 3 2026, 16:04:56) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
=== RESTART: C:/Users/chall/AppData/Local/Programs/Python/Python314/co5(4).py ===
TP = 5
TN = 3
FP = 1
FN = 1

Accuracy = 0.8
Precision = 0.83
Recall = 0.83
F1-Score = 0.83
>>>
```

5. Design pseudocode for comparing kNN, Decision Tree, Logistic Regression, and K-Means models using appropriate evaluation measures and selecting the most suitable model for a given Data Science problem.

Problem Statement

Design and implement a program to compare k-Nearest Neighbors, Decision Tree, Logistic Regression, and K-Means models using appropriate evaluation measures and select the most suitable model for a given Data Science problem.

Algorithm

1. Start.
2. Import the required libraries.
3. Load the dataset.
4. Separate input features and target values.
5. Split the data into training and testing datasets.
6. Standardize the feature values.
7. Train the kNN model.
8. Train the Decision Tree model.

9. Train the Logistic Regression model.
10. Predict the test data using each model.
11. Calculate Accuracy, Precision, Recall, and F1-Score.
12. Apply K-Means clustering.
13. Calculate the Silhouette Score for K-Means.
14. Compare the performance values.
15. Select the model with the best suitable performance.
16. Display the results.
17. Stop.

Pseudocode

BEGIN

LOAD dataset

Separate features X and target y

Split dataset into training and testing data

Standardize feature values

Create kNN model

Train kNN model

Predict test data

Calculate Accuracy, Precision, Recall and F1-Score

Create Decision Tree model

Train Decision Tree model

Predict test data

Calculate Accuracy, Precision, Recall and F1-Score

Create Logistic Regression model

Train Logistic Regression model

Predict test data

Calculate Accuracy, Precision, Recall and F1-Score

Apply K-Means clustering

Calculate Silhouette Score

Compare evaluation results

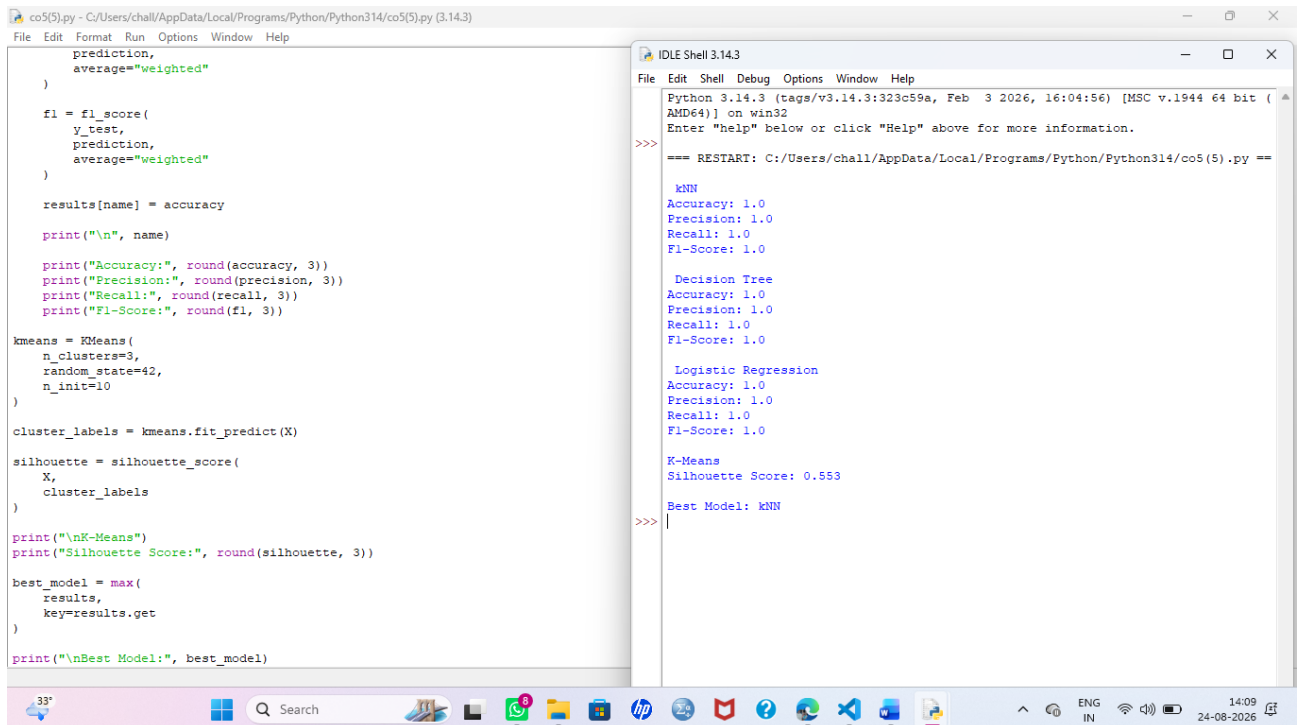
Select the model with best performance

DISPLAY all model results

DISPLAY best model

END

Output



The screenshot shows a Python IDE with two windows. The left window displays a Python script for evaluating models. The right window shows the output of the script.

```
co5(5).py - C:/Users/chall/AppData/Local/Programs/Python/Python314/co5(5).py (3.14.3)
File Edit Format Run Options Window Help

    prediction,
    average="weighted"
)

f1 = f1_score(
    y_test,
    prediction,
    average="weighted"
)

results[name] = accuracy

print("\n", name)

print("Accuracy:", round(accuracy, 3))
print("Precision:", round(precision, 3))
print("Recall:", round(recall, 3))
print("F1-Score:", round(f1, 3))

kmeans = KMeans(
    n_clusters=3,
    random_state=42,
    n_init=10
)

cluster_labels = kmeans.fit_predict(X)

silhouette = silhouette_score(
    X,
    cluster_labels
)

print("\nK-Means")
print("Silhouette Score:", round(silhouette, 3))

best_model = max(
    results,
    key=results.get
)

print("\nBest Model:", best_model)
```

```
IDLE Shell 3.14.3
File Edit Shell Debug Options Window Help

Python 3.14.3 (tags/v3.14.3:323c59a, Feb  3 2026, 16:04:56) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
=== RESTART: C:/Users/chall/AppData/Local/Programs/Python/Python314/co5(5).py ==
    kNN
Accuracy: 1.0
Precision: 1.0
Recall: 1.0
F1-Score: 1.0

    Decision Tree
Accuracy: 1.0
Precision: 1.0
Recall: 1.0
F1-Score: 1.0

    Logistic Regression
Accuracy: 1.0
Precision: 1.0
Recall: 1.0
F1-Score: 1.0

    K-Means
Silhouette Score: 0.553

Best Model: kNN
>>>
```

The taskbar at the bottom shows the system clock as 14:09 on 24-08-2026, with the language set to ENG IN.